СОДЕРЖАНИЕ

1 НАЗНАЧЕНИЕ ПРОГРАММЫ 3

1.1 Общие сведения 3

1.2 Назначение программы 3

1.3 Основные функции программы 4

1.4 Область применения 4

1.5 Особенности реализации и ограничения 4

2 УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ 6

2.1 Требования к техническим средствам 6

2.2 Требования к программным средствам 6

2.3 Требования к персоналу 8

2.4 Состав поставки 8

3 ВЫПОЛНЕНИЕ ПРОГРАММЫ 10

3.1 Общий порядок выполнения работ 10

3.2 Установка интерпретатора Python 10

3.3 Установка сервера PostgreSQL 11

3.4 Установка библиотек Python 12

3.5 Создание базы данных 12

3.6 Настройка параметров программы 13

3.7 Запуск программы 14

3.8 Проверка результатов выполнения 15

3.9 Повторный запуск и работа со снимком 16

3.10 Выполнение программы в консольном режиме 17

3.11 Завершение работы программы 17

3.12 Чек-лист выполнения программы 18
4 СООБЩЕНИЯ ПРОГРАММИСТУ 19
5 СОСТАВ КЛАССОВ И ФУНКЦИЙ 21
5.1 Общая структура модуля 21
5.2 Класс FileStatus 22
5.3 Класс FileRecord 22
5.4 Класс ConsoleStorage 23
5.5 Класс DatabaseStorage 24
5.6 Функции уровня модуля 25
5.7 Функция compute_md5 26
5.8 Функция convert_office_file 26
5.9 Функция read_text_content 27
5.10 Функция build_record 28
5.11 Функция load_snapshot 28
5.12 Функция save_snapshot 28
5.13 Функция scan_directory 29
5.14 Функция run_scan 29
5.15 Функция main 30
5.16 Порядок взаимодействия функций и классов 30
6 ТЕСТОВЫЕ ПРИМЕРЫ 32
6.1 Назначение и методика тестирования 32
6.2 Тестовый набор 33
6.3 Тестовые примеры 34
6.4 Сводные результаты тестовых примеров 40

1 НАЗНАЧЕНИЕ ПРОГРАММЫ

1.1 Общие сведения

Программа реализована на языке Python 3 в виде одного модуля main.py и входит в состав комплекса «scanner». Обозначение документа — scanner.00001-01 33 01. Программа не имеет графического интерфейса и управляется значениями констант, объявленных в начале модуля, и переменными окружения.

Программа применяется совместно с СУБД PostgreSQL: результаты сканирования записываются в базу данных file_storage, а также в файл снимка snapshot.json, который используется при следующем запуске для определения статусов файлов.

Настоящий документ оформлен в соответствии с требованиями ГОСТ 19.504-79 «Единая система программной документации. Руководство программиста. Требования к содержанию и оформлению» и ГОСТ 2.105-95 «Единая система конструкторской документации. Общие требования к текстовым документам».

1.2 Назначение программы

Программа предназначена для инвентаризации содержимого файлового хранилища: последовательного обхода заданного каталога, вычисления контрольной суммы MD5 каждого файла, извлечения текстового содержимого и метаданных, определения статуса файла по отношению к предыдущему сканированию и сохранения полученных сведений в PostgreSQL и в файл snapshot.json.

Информация, достаточная для понимания функций программы: результатом одного запуска является набор записей о файлах, каждая из которых содержит полный путь, имя, расширение, размер, даты создания и изменения, момент сканирования, контрольную сумму MD5, извлечённое содержимое и статус. Совокупность записей одного запуска образует скан, которому в базе данных соответствует строка таблицы scans.

1.3 Основные функции программы

Программа выполняет следующие функции:

1) рекурсивный обход каталога, заданного параметром ROOT_PATH, с отбором только файлов (каталоги пропускаются);

2) вычисление контрольной суммы MD5 потоковым чтением файла блоками по 64 КБ (константа HASH_CHUNK_SIZE);

3) извлечение текстового содержимого для текстовых форматов, документов Word, Excel и PowerPoint, PDF, CSV, YAML, HTML, ODF, а также метаданных изображений, аудиофайлов, архивов, баз SQLite и прочих форматов;

4) определение статуса файла относительно предыдущего снимка: new, unchanged, modified, deleted;

5) сохранение результатов в PostgreSQL в таблицы scans, paths, files, json либо, в консольном режиме, вывод карточек файлов в стандартный поток вывода;

6) формирование файла снимка snapshot.json для следующего запуска;

7) вывод итоговой сводки по количеству обработанных файлов каждого статуса.

1.4 Область применения

Программа применяется при инвентаризации файловых хранилищ, контроле изменений в каталогах с документацией, подготовке метаданных для последующего поиска и анализа, а также при наполнении базы данных сведениями о файлах. Программа рассчитана на однопользовательский режим работы на локальном рабочем месте и не предназначена для одновременного сканирования одного каталога несколькими экземплярами.

1.5 Особенности реализации и ограничения

При эксплуатации программы необходимо учитывать следующие особенности, обусловленные принятыми в коде решениями:

1) **Сравнение выполняется по полному пути файла.** Ключом снимка является строка path, поэтому переименование или перемещение файла приводит к появлению записи со статусом new и записи со статусом deleted, даже если содержимое не изменилось.

2) **MD5 используется как средство обнаружения изменений, а не как средство защиты.** Алгоритм не является криптостойким и не должен применяться для подтверждения подлинности файлов.

3) **Файлы размером более 5 МБ (MAX_CONTENT_SIZE) не анализируются на содержимое.** Для них вычисляются размер, даты и MD5, а поле content остаётся пустым (NULL).

4) **Файлы .doc и .xls обрабатываются только при установленном Microsoft Office и библиотеке pywin32.** В остальных случаях выводится сообщение [CONVERT-SKIP], работа программы продолжается.

5) **Чтение архивов .rar требует наличия внешнего распаковщика UnRAR или 7-Zip.** Без него выводится сообщение [CONTENT-SKIP].

6) **Обход каталога выполняется последовательно в одном потоке.** Время сканирования пропорционально суммарному объёму файлов, поскольку MD5 вычисляется для каждого файла при каждом запуске.

7) **Запись со статусом deleted не удаляется из таблицы files.** Строка помечается статусом deleted, поля last_scan_id и scanned_at обновляются, благодаря чему сохраняется история.

8) **Уникальность файла определяется парой (path_id, name).** В одной папке не может быть двух записей с одинаковым именем; ограничение закреплено в схеме базы данных.

9) **Снимок snapshot.json содержит только существующие файлы.** Записи со статусом deleted в файл не попадают, поэтому после удаления файла повторное сообщение о нём не формируется.

10) **Кодировка текстовых файлов определяется автоматически (chardet).** Если определить кодировку не удалось, применяется UTF-8 с заменой некорректных байтов, а символ NUL удаляется из текста.

2 УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ

2.1 Требования к техническим средствам

Программа выполняется на персональном компьютере под управлением операционной системы семейства Windows. Минимальный и рекомендуемый состав технических средств приведён в таблице 1.

Таблица 1 – Требования к техническим средствам

Параметр	Минимальные значения	Рекомендуемые значения
Процессор	двухъядерный, 1,6 ГГц	четырёхъядерный, 2,4 ГГц и выше
Оперативная память	2 ГБ	8 ГБ и выше
Свободное место на диске	не менее 1 ГБ, помимо места под базу данных	не менее 10 ГБ с учётом роста базы данных
Разрешение экрана	1024 × 768	1920 × 1080
Свободный порт	TCP 5432 (PostgreSQL)	TCP 5432
Привод и устройства	клавиатура, манипулятор «мышь»	клавиатура, манипулятор «мышь»
Канал связи	не требуется	требуется только для установки библиотек из сети Интернет

Объём оперативной памяти определяется не размером сканируемого каталога, а размером отдельных файлов: содержимое файла читается целиком только для файлов не более 5 МБ, а контрольная сумма вычисляется блоками по 64 КБ. Место на диске расходуется на базу данных file_storage, в которой хранится извлечённый текст, и на файл snapshot.json, содержащий те же сведения в формате JSON.

2.2 Требования к программным средствам

Для выполнения программы необходимы операционная система, интерпретатор Python, сервер PostgreSQL, клиент PostgreSQL и перечисленные ниже библиотеки. Состав программных средств приведён в таблице 2; версии соответствуют файлу requirements.txt поставки.

Таблица 2 – Перечень программных средств

1	Windows 10 / Windows 11 (64-разрядная)	10, 11	Операционная система, под управлением которой выполняется программа
2	Python	3.9 и выше	Интерпретатор языка, среда выполнения модуля main.py
3	PostgreSQL	13 и выше	СУБД, в которой размещается база данных file_storage
4	psycopg2-binary	2.9.13	Драйвер подключения к PostgreSQL, используется модулем DatabaseStorage
5	chardet	7.6.0	Автоматическое определение кодировки текстовых файлов и CSV
6	PyMuPDF (модуль fitz)	1.28.2	Извлечение текста из документов PDF
7	py7zr	1.1.3	Чтение архивов 7z, получение списка файлов
8	rarfile	4.5	Чтение архивов rar
9	PyYAML	6.0.3	Разбор и повторная сериализация файлов yaml и yml
10	beautifulsoup4	4.15.0	Извлечение текста из HTML без разметки
11	python-docx	1.2.0	Чтение документов docx
12	hachoir	3.4.0	Универсальное извлечение метаданных из файлов неизвестных форматов
13	mutagen	1.48.1	Чтение тегов аудиофайлов
14	odfpy	1.4.1	Чтение документов ODF (odt, ods, odp)
15	openpyxl	3.1.5	Чтение таблиц xlsx
16	Pillow	12.3.0	Определение формата, размеров и цветовой модели изображений
17	python-pptx	1.0.2	Чтение презентаций pptx
18	olefile	без фиксации версии	Разбор OLE-контейнеров, в том числе презентаций ppt
19	pywin32	без фиксации версии	Управление приложениями Microsoft Office через COM при конвертации doc и xls
20	pgAdmin 4 либо psql	4.x	Клиент PostgreSQL для выполнения скрипта create_database.sql и проверки данных
21	UnRAR или 7-Zip	—	Внешний распаковщик, используемый библиотекой rarfile при чтении архивов rar
22	Microsoft Office (Word, Excel)	2010 и выше	Требуется только для конвертации файлов doc и xls

Модули стандартной библиотеки Python (hashlib, csv, json, sqlite3, zipfile, io, os, sys, pathlib, datetime, dataclasses, typing) устанавливать отдельно не требуется: они входят в состав интерпретатора.

Библиотеки pywin32, olefile, UnRAR и Microsoft Office нужны только для работы с соответствующими форматами файлов. При их отсутствии программа сохраняет работоспособность: файл обрабатывается частично, а в

консоль выводится сообщение, начинающееся с [CONVERT-SKIP] или [CONTENT-SKIP].

2.3 Требования к персоналу

К выполнению программы допускается программист, владеющий следующими знаниями и навыками:

1) основы языка Python в объёме, достаточном для чтения и редактирования констант модуля и анализа сообщений интерпретатора;

2) работа в командной строке Windows (переход в каталог, запуск интерпретатора, установка пакетов);

3) основы языка SQL и умение работать с клиентом PostgreSQL (pgAdmin 4 или psql): выполнение скрипта, просмотр таблиц, выполнение запросов SELECT;

4) представление о файловых системах, расширениях файлов, кодировках текста и контрольных суммах;

5) права администратора для установки Python, PostgreSQL и библиотек, а также права на чтение сканируемого каталога и запись в каталог проекта.

2.4 Состав поставки

В состав поставки входят файлы, перечисленные в таблице 3. Все файлы размещаются в одном каталоге проекта.

Таблица 3 – Состав поставки

Файл (каталог)	Назначение
main.py	Основной модуль программы: конфигурация, классы хранилищ, функции извлечения содержимого и логика сканирования
create_database.sql	Скрипт создания базы данных file_storage и таблиц scans, paths, files, json
requirements.txt	Перечень библиотек Python с зафиксированными версиями
snapshot.json	Снимок предыдущего сканирования; при первом запуске может отсутствовать
create_test_files.py	Вспомогательный скрипт подготовки тестового набора файлов (не входит в контур программы)

Файл (каталог)	Назначение
png	Каталог с иллюстрациями: схема базы данных и снимки экрана клиента PostgreSQL

Файл snapshot.json создаётся программой автоматически, если он отсутствует. Каталог png не используется при выполнении программы и входит в поставку в качестве иллюстративного материала.

3 ВЫПОЛНЕНИЕ ПРОГРАММЫ

3.1 Общий порядок выполнения работ

Выполнение программы включает перечисленные ниже работы, которые необходимо провести в указанной последовательности:

1) установить интерпретатор Python и убедиться в доступности команды python (подраздел 3.2);

2) установить сервер PostgreSQL и клиент pgAdmin 4, проверить состояние службы (подраздел 3.3);

3) установить библиотеки Python, перечисленные в таблице 2 (подраздел 3.4);

4) создать базу данных file_storage и таблицы, выполнив скрипт create_database.sql (подраздел 3.5);

5) настроить параметры программы: строку подключения, сканируемый каталог, режим сохранения (подраздел 3.6);

6) запустить программу на выполнение (подраздел 3.7);

7) проверить содержимое таблиц базы данных и файла snapshot.json (подраздел 3.8).

При повторных запусках работы по пунктам 1)–5 выполняются только один раз; дальнейшая эксплуатация сводится к запуску программы и проверке результатов.

3.2 Установка интерпретатора Python

1) Запустить загруженный установщик интерпретатора (например, python-3.13.0-amd64.exe).

2) В первом окне установщика установить флажок «Add python.exe to PATH», иначе команда python не будет доступна в командной строке.

3) Нажать «Install Now» и дождаться завершения установки, затем закрыть установщик.

4) Открыть командную строку: нажать Win+R, ввести cmd и нажать Enter.

5) Проверить установку командой, приведённой ниже. Команда должна вывести версию интерпретатора не ниже 3.9.

```
python --version
```

Ожидаемый вывод: Python 3.13.0 (номер версии может отличаться). Если вместо версии открывается Microsoft Store или выводится сообщение «python не является внутренней или внешней командой», следует использовать команду py --version либо переустановить интерпретатор с флажком «Add python.exe to PATH».

3.3 Установка сервера PostgreSQL

1) Запустить установщик PostgreSQL (например, postgresql-16.x-windows-x64.exe).

2) В списке компонентов оставить «PostgreSQL Server», «pgAdmin 4» и «Command Line Tools».

3) Задать пароль пользователя postgres и запомнить его: пароль потребуется при настройке строки подключения.

4) Оставить порт по умолчанию 5432; если порт занят, запомнить выбранный, так как он указывается в строке подключения.

5) Завершить установку, сняв флажок запуска Stack Builder.

6) Убедиться, что служба PostgreSQL запущена. Проверка выполняется в оснастке «Службы» (services.msc) или командой, приведённой ниже; имя службы зависит от версии сервера.

```
sc query postgresql-x64-16 | findstr STATE
```

Ожидаемый вывод содержит строку STATE : 4 RUNNING. Дополнительно можно проверить подключение к серверу:

```
psql -U postgres -h localhost -p 5432 -c "SELECT version();"
```

3.4 Установка библиотек Python

Открыть командную строку, перейти в каталог проекта и установить библиотеки, перечислив пакеты, входящие в поставку:

```
cd C:\scaner

python -m pip install beautifulsoup4 chardet hachoir mutagen odfpy olefile openpyxl
pillow psycopg2-binary py7zr pymupdf python-docx python-pptx pyyaml rarfile pywin32
```

Если источником перечня зависимостей служит файл requirements.txt, установка выполняется командой:

```
python -m pip install -r requirements.txt
```

При отсутствии доступа к сети Интернет установка выполняется из локального каталога с заранее загруженными пакетами формата whl:

```
python -m pip install --no-index --find-links=.\packages psycopg2-binary chardet
pymupdf py7zr rarfile pyyaml beautifulsoup4 python-docx hachoir mutagen odfpy openpyxl
pillow python-pptx olefile pywin32
```

Проверка установки выполняется командой, приведённой ниже. Если какой-либо модуль отсутствует, команда завершится с ошибкой ModuleNotFoundError, в тексте которой будет указано имя модуля.

```
python -c "import psycopg2, fitz, docx, openpyxl, pptx, PIL, chardet, yaml, bs4,
py7zr, rarfile, mutagen, odf, hachoir, olefile"
```

Отсутствие вывода означает, что все модули установлены. Библиотека pywin32 проверяется отдельно, так как требуется только для конвертации файлов doc и xls:

```
python -c "import win32com.client; print('pywin32 OK')"
```

3.5 Создание базы данных

1) Открыть pgAdmin 4 и подключиться к серверу PostgreSQL, введя пароль пользователя postgres.

2) В дереве объектов выбрать служебную базу postgres, открыть инструмент «Query Tool» (сервис → Query Tool).

3) Открыть файл create_database.sql (кнопка открытия файла) и выполнить только первую часть скрипта — команду CREATE DATABASE file_storage (клавиша F5).

4) Обновить дерево объектов, убедиться, что база file_storage создана, и открыть для неё новый «Query Tool».

5) Выполнить вторую часть скрипта: создание таблиц scans, paths, files, json, индекса idx_paths_parent и миграционной команды ALTER TABLE.

6) Выполнить третью часть скрипта — проверочные запросы. Первый запрос должен вернуть четыре строки: files, json, paths, scans.

Если база данных уже создавалась и её требуется пересоздать, перед первой частью скрипта выполнить в базе postgres команду, приведённую ниже (команда удаляет базу вместе с содержимым, поэтому применять её следует только при необходимости полного пересоздания):

```
DROP DATABASE IF EXISTS file_storage WITH (FORCE);
```

Структура созданных таблиц приведена в приложении А.

3.6 Настройка параметров программы

Перед первым запуском открыть файл main.py текстовым редактором и проверить значения констант, перечисленных в таблице 4. Константы объявлены в начале модуля, в разделе «КОНФИГ».

Таблица 4 – Параметры настройки программы

Параметр	Значение в поставке	Назначение
ENABLE_DB_SYNC	True	Режим сохранения результатов: True — запись в PostgreSQL, False — вывод карточек файлов в консоль без обращения к базе
DB_DSN	переменная окружения DB_DSN; при её отсутствии — строка подключения к локальной базе file_storage	Строка подключения к PostgreSQL в формате libpq: host, port, dbname, user и пароль пользователя
ROOT_PATH	E:\MAX_files	Каталог, который подвергается сканированию; обход выполняется рекурсивно
SNAPSHOT_FILE	snapshot.json	Путь к файлу снимка; относительный путь означает каталог, из которого запущена программа

Параметр	Значение в поставке	Назначение
HASH_CHUNK_SIZE	65536 (64 КБ)	Размер блока при потоковом вычислении MD5
MAX_CONTENT_SIZE	5242880 (5 МБ)	Предельный размер файла, для которого выполняется извлечение содержимого
TEXT_EXTENSIONS	.txt, .py, .json, .md, .log, .xml	Расширения, обрабатываемые как простой текст с автоопределением кодировки

Строку подключения рекомендуется задавать через переменную окружения, чтобы не хранить реквизиты доступа в исходном коде. Переменная читается при загрузке модуля, поэтому её необходимо установить до запуска программы. Пароль в строку подключения не включается: драйвер psycopg2, использующий библиотеку libpq, читает его из отдельной переменной окружения PGPASSWORD. В командной строке строка подключения задаётся командой:

```
set DB_DSN=host=localhost port=5432 dbname=file_storage user=postgres
```

Перед запуском переменной окружения PGPASSWORD присваивается пароль пользователя postgres, указанный при установке сервера; делается это стандартной командой set. В оболочке PowerShell используется иной синтаксис:

```
$env:DB_DSN = "host=localhost port=5432 dbname=file_storage user=postgres"
```

Переменная PGPASSWORD в PowerShell задаётся так же — через префикс $env; после закрытия окна значения переменных теряются, поэтому для постоянного хранения используется команда setx.

Для постоянного использования переменную можно сохранить командой setx DB_DSN "..." (после этого требуется открыть новое окно командной строки). Значение ROOT_PATH задаётся в коде в виде строкового литерала, например r"E:\MAX_files"; каталог должен существовать и быть доступным для чтения.

3.7 Запуск программы

1) Открыть командную строку.

2) Перейти в каталог проекта командой cd с полным путём к каталогу, в котором размещён файл main.py.

3) При необходимости установить переменную окружения DB_DSN (подраздел 3.6).

4) Запустить программу командой, приведённой ниже.

5) Дождаться завершения работы: признаком служит итоговая сводка, ограниченная линиями из символов «=».

```
cd C:\scaner
python main.py
```

В процессе работы для каждого файла выводится строка с русской подписью статуса и полным путём, например:

```
[НОВЫЙ] E:\MAX_files\docs\report.txt
[НОВЫЙ] E:\MAX_files\tables\data.csv
[БЕЗ ИЗМЕНЕНИЙ] E:\MAX_files\media\logo.png
```

После обхода каталога выводится итоговая сводка:

```
============================================================
Всего обработано:     9
  новых:              9
  без изменений:      0
  изменено:           0
  удалено:            0
============================================================
```

Значение «Всего обработано» равно сумме четырёх счётчиков и включает записи-маркеры удалённых файлов. Нормальное завершение сопровождается кодом возврата 0; проверить код можно командой echo %ERRORLEVEL% сразу после запуска.

3.8 Проверка результатов выполнения

Результаты проверяются в клиенте PostgreSQL и в файле снимка. Перечень выполняемых действий:

1) Открыть pgAdmin 4, подключиться к базе file_storage и открыть «Query Tool».

2) Выполнить запрос к таблице scans (запрос 1 приведён ниже). В последней строке поле finished_at должно быть заполнено, а поле files_total — равно сумме files_new, files_unchanged, files_modified и files_deleted.

3) Проверить число папок в таблице paths и распределение файлов по статусам в таблице files (запросы 2 и 3).

4) Убедиться, что для каждого файла сформирована JSON-запись в таблице json (запрос 4).

5) Убедиться, что в каталоге проекта обновлён файл snapshot.json и число записей в нём равно числу существующих файлов.

```
-- запрос 1: сведения о последнем скане
SELECT id, root_path, started_at, finished_at, files_total,
       files_new, files_unchanged, files_modified, files_deleted
  FROM scans
 ORDER BY id DESC
 LIMIT 1;

-- запрос 2: количество папок
SELECT count(*) AS folders FROM paths;

-- запрос 3: распределение файлов по статусам
SELECT status, count(*) AS cnt
  FROM files
 GROUP BY status
 ORDER BY status;

-- запрос 4: содержимое JSON-метаданных
SELECT f.name, f.status, f.change_count,
       j.data->>'md5_hash' AS md5,
       left(j.data->>'content', 60) AS content_head
  FROM files f
  JOIN json j ON j.file_id = f.id
 ORDER BY f.name
 LIMIT 10;
```

Ожидаемые результаты: запрос 1 возвращает одну строку с заполненным finished_at; запрос 3 возвращает строки только для тех статусов, которые встретились при сканировании; запрос 4 возвращает для каждого файла значение md5 из 32 шестнадцатеричных символов.

3.9 Повторный запуск и работа со снимком

При повторном запуске программа читает файл snapshot.json, сопоставляет полные пути файлов и сравнивает контрольные суммы MD5. Файлы, содержимое которых не изменилось, получают статус unchanged; изменённые — modified; отсутствующие в снимке — new; отсутствующие на диске, но имеющиеся в снимке — deleted. По завершении сканирования файл снимка перезаписывается целиком, при этом записи со статусом deleted в него не включаются.

Для получения корректных статусов необходимо запускать программу из одного и того же каталога: путь к файлу снимка задан относительным (snapshot.json), поэтому при запуске из другого каталога снимок считается отсутствующим и все файлы получают статус new.

3.10 Выполнение программы в консольном режиме

Для проверки работы без базы данных следует установить значение False для константы ENABLE_DB_SYNC. В этом режиме создаётся объект ConsoleStorage, подключение к PostgreSQL не выполняется, а для каждого файла печатается карточка, содержащая русскую подпись статуса, технический статус, путь, размер, MD5 и даты в местном времени. Пример карточки приведён ниже.

```
[НОВЫЙ] report.txt
    status:      new
    путь:        E:\MAX_files\docs\report.txt
    размер:      115 B
    md5:         d50a4f92748a685f564d2e37dab5b3b9
    создан:      2026-10-20 22:13:03
    изменён:     2026-10-20 22:13:10
```

Консольный режим применяется при проверке логики определения статусов, при отладке и при отсутствии доступа к серверу PostgreSQL. Файл snapshot.json в этом режиме также создаётся и обновляется.

3.11 Завершение работы программы

После вывода итоговой сводки программа вызывает метод close(), который фиксирует транзакцию (commit) и закрывает соединение с базой данных. При возникновении исключения в ходе сканирования вызывается метод rollback(): незавершённая транзакция отменяется, соединение закрывается, а исключение передаётся вызывающему коду, поэтому программа завершается с ненулевым кодом возврата и трассировкой в стандартном потоке ошибок.

Прерывание работы сочетанием клавиш Ctrl+C не перехватывается программой. При этом незафиксированная транзакция отменяется сервером PostgreSQL после разрыва соединения, поэтому частично записанный скан в

базе данных не остаётся. Файл snapshot.json в случае прерывания не изменяется, так как записывается только после успешного завершения сканирования.

3.12 Чек-лист выполнения программы

Перед началом эксплуатации рекомендуется проверить выполнение условий, перечисленных в таблице 5.

Таблица 5 – Чек-лист выполнения программы

№	Проверяемое условие	Признак выполнения
1	Интерпретатор Python установлен	команда python --version выводит версию 3.9 или выше
2	Сервер PostgreSQL установлен и запущен	служба postgresql-x64-* находится в состоянии RUNNING
3	Библиотеки Python установлены	команда проверки импорта завершается без сообщений об ошибке
4	База данных и таблицы созданы	запрос к information_schema возвращает таблицы files, json, paths, scans
5	Параметры DB_DSN и ROOT_PATH настроены	строка подключения соответствует серверу, каталог существует
6	Программа запускается без необработанных исключений	в консоли выведена итоговая сводка, код возврата равен 0
7	Результаты записаны в базу данных	в таблице scans заполнено поле finished_at, таблица files не пуста
8	Файл snapshot.json создан	файл существует в каталоге запуска и содержит массив записей

4 СООБЩЕНИЯ ПРОГРАММИСТУ

Сообщения выводятся в стандартный поток вывода функцией print; сообщения интерпретатора Python и трассировки исключений выводятся в стандартный поток ошибок. Формат сообщений программы: квадратные скобки с кратким кодом, затем имя файла или путь. Перечень сообщений, их причины и действия программиста приведены в таблице 6.

Таблица 6 – Сообщения программисту

Сообщение	Причина	Действия программиста
[НОВЫЙ] <путь> [БЕЗ ИЗМЕНЕНИЙ] <путь> [ИЗМЕНЁН] <путь> [УДАЛЁН] <путь>	Информационное сообщение о статусе очередного файла (метод save хранилища)	Действий не требуется; используется для контроля хода сканирования
[SKIP] <путь> -> <Тип ошибки>: <текст>	Файл не удалось прочитать при получении сведений: нет прав доступа, файл занят другой программой, путь слишком длинный	Закрыть файл в приложении, проверить права на чтение, сократить путь; запись о файле в этом скане не создаётся
[CONTENT-SKIP] <имя>: <Тип ошибки>: <текст>	Не удалось извлечь текст или метаданные: файл повреждён, формат не поддерживается библиотекой, отсутствует внешний распаковщик для rar	Проверить файл вручную; запись сохраняется, поле content остаётся пустым. Для rar установить UnRAR или 7-Zip
[CONVERT-SKIP] <имя>: <Тип ошибки>: <текст>	Не удалось конвертировать doc или xls: не установлен Microsoft Office или библиотека pywin32, файл защищён паролем	Установить Office и pywin32 либо сохранить копию файла в формате docx или xlsx вручную
[КОНВЕРТАЦИЯ] <имя> -> <новое имя>	Информационное сообщение: рядом с исходным файлом создана копия в современном формате	Действий не требуется; копия появляется в том же каталоге и при следующем скане будет обработана как отдельный файл
[WARN] Не удалось прочитать снимок: <текст>	Файл snapshot.json повреждён или не соответствует формату JSON	Проверить файл; при необходимости удалить его — сканирование выполнится, но все файлы получат статус new
Путь не существует: <путь>	Каталог, указанный в ROOT_PATH, отсутствует или недоступен	Исправить значение ROOT_PATH; запись в таблице scans при этом не создаётся, код возврата равен 0
psycopg2.OperationalError: connection to server at "localhost", port 5432 failed	Сервер PostgreSQL не запущен, неверны порт, имя базы или пароль	Запустить службу PostgreSQL, проверить значения DB_DSN и доступность порта 5432
psycopg2.errors.UndefinedTable: relation "scans" does not exist	Не выполнена вторая часть скрипта create_database.sql	Подключиться к базе file_storage и выполнить вторую часть скрипта
psycopg2.errors.InvalidTextRepresentation либо ValueError: A string literal cannot contain NUL (0x00) character	В извлечённом тексте присутствует символ NUL либо недопустимая последовательность байтов	Проверить файл; в методе save символ NUL удаляется автоматически, поэтому сообщение свидетельствует об иной причине и требует анализа файла

Сообщение	Причина	Действия программиста
psycopg2.errors.StringDataRightTruncation: value too long for type character varying(255)	Имя файла или папки превышает 255 символов, установленных схемой базы данных	Сократить имя файла или папки либо увеличить длину поля в схеме базы данных
ModuleNotFoundError: No module named '<имя модуля>'	Библиотека не установлена в используемом интерпретаторе	Выполнить установку библиотек (подраздел 3.4) и повторить запуск
UnicodeEncodeError: 'charmap' codec can't encode characters	Кодировка консоли не соответствует выводимому тексту (возникает при перенаправлении вывода)	Выполнить chcp 65001 перед запуском либо перенаправить вывод в файл с указанием кодировки UTF-8
Итоговая сводка (линии из символов «=» и счётчики по статусам)	Информационное сообщение о завершении сканирования	Действий не требуется; счётчики используются при проверке результатов (подраздел 3.8)

Сообщения, начинающиеся с [CONTENT-SKIP], [CONVERT-SKIP] и [SKIP], не прерывают работу программы: необработанный файл пропускается, сканирование продолжается. Сообщения интерпретатора, приведённые в последних строках таблицы, не перехватываются программой и требуют вмешательства программиста.

5 СОСТАВ КЛАССОВ И ФУНКЦИЙ

5.1 Общая структура модуля

Программа состоит из одного модуля main.py, который построен по принципу разделения ответственности: классы отвечают за хранение результатов, функции уровня модуля — за извлечение данных, определение статусов и управление сканированием. Модуль включает следующие составные части, расположенные в порядке объявления:

1) блок импорта: модули стандартной библиотеки и внешние библиотеки (psycopg2, fitz, py7zr, rarfile, yaml, bs4, docx, openpyxl, pptx, PIL, mutagen, odf, hachoir);

2) раздел «КОНФИГ»: константы ENABLE_DB_SYNC, DB_DSN, ROOT_PATH, SNAPSHOT_FILE, HASH_CHUNK_SIZE, MAX_CONTENT_SIZE, TEXT_EXTENSIONS и настройка кодировки стандартного вывода;

3) классы FileStatus, FileRecord, ConsoleStorage, DatabaseStorage;

4) выбор объекта хранилища по значению ENABLE_DB_SYNC: storage = DatabaseStorage() при значении True, storage = ConsoleStorage() при значении False;

5) функции compute_md5, convert_office_file, read_text_content, build_record, load_snapshot, save_snapshot, scan_directory, run_scan, main;

6) точка входа: блок if __name__ == "__main__", вызывающий main().

Классы ConsoleStorage и DatabaseStorage имеют одинаковый набор методов (begin_scan, save, finish_scan, rollback, close), поэтому функции run_scan и main работают с любым из них без проверки типа. Такое совместимое поведение реализует принцип подстановки: замена хранилища выполняется одной константой ENABLE_DB_SYNC.

5.2 Класс FileStatus

Класс FileStatus хранит набор строковых констант, обозначающих статус файла при сравнении с прошлым снимком, а также служебные атрибуты для перебора и вывода. Класс не создаёт экземпляров: используются только его атрибуты.

Таблица 7 – Константы и атрибуты класса FileStatus

Атрибут	Значение	Назначение
NEW	"new"	Файл встречен впервые: его пути нет в предыдущем снимке
UNCHANGED	"unchanged"	Контрольная сумма MD5 совпала со снимком, содержимое не изменилось
MODIFIED	"modified"	Контрольная сумма MD5 отличается от значения в снимке
DELETED	"deleted"	Файл присутствовал в снимке, но отсутствует на диске
ALL	кортеж из четырёх констант	Упорядоченный неизменяемый набор всех статусов для перебора и инициализации счётчиков
RU	словарь	Русские подписи статусов: НОВЫЙ, БЕЗ ИЗМЕНЕНИЙ, ИЗМЕНЁН, УДАЛЁН

Атрибут RU используется при выводе в консоль и при формировании строки прогресса в методе save хранилища; при отсутствии значения в словаре выводится сам технический статус.

5.3 Класс FileRecord

Класс FileRecord описывает одну запись о файле и содержит все атрибуты, которые программа сохраняет для каждого файла. Класс объявлен с декоратором @dataclass, поэтому конструктор, метод сравнения и метод asdict формируются автоматически. Состав атрибутов приведён в таблице 8.

Таблица 8 – Атрибуты класса FileRecord

Атрибут	Тип	Назначение	Значение по умолчанию
path	str	Полный путь к файлу (результат resolve)	обязательный
name	str	Имя файла с расширением	обязательный
extension	str	Расширение в нижнем регистре	обязательный
size_bytes	int	Размер файла в байтах	обязательный
created_at	datetime	Дата создания файла в UTC	обязательный

Атрибут	Тип	Назначение	Значение по умолчанию
modified_at	datetime	Дата последнего изменения в UTC	обязательный
status	str	Статус файла	FileStatus.NEW
scanned_at	datetime	Момент сканирования в UTC	текущее время (datetime.now)
content	Optional[str]	Извлечённый текст или None	None
md5_hash	Optional[str]	Контрольная сумма MD5 или None	None

Атрибуты, имеющие значения по умолчанию, объявлены после обязательных: это требование синтаксиса Python для классов данных. Состав атрибутов соответствует набору сведений, сохраняемых для файла в таблицах files и json.

Методы класса:

1) to_jsonable() — превращает запись в словарь для сохранения в snapshot.json и в таблицу json. Даты created_at, modified_at и scanned_at переводятся в текст формата ISO 8601 (метод isoformat), остальные значения сохраняются без изменений.

2) to_row() — собирает словарь значений для SQL-запроса. Ключи словаря совпадают с именами параметров запроса: name, extension, size_bytes, md5_hash, created_at, modified_at, scanned_at, content, status. Атрибут path в словарь не входит: каталог файла передаётся отдельно, через идентификатор папки, а идентификатор скана добавляется в методе save.

5.4 Класс ConsoleStorage

Класс ConsoleStorage — хранилище, которое выводит результаты сканирования в консоль. Применяется при ENABLE_DB_SYNC = False, а также при отладке. Методы класса приведены в таблице 9.

Таблица 9 – Методы класса ConsoleStorage

Метод	Назначение
save(record)	Печатает карточку файла: русскую подпись статуса, имя, технический статус, путь, размер, MD5, дату создания и дату изменения. Даты приводятся к местному времени в формате ГГГГ-ММ-ДД ЧЧ:ММ:СС
begin_scan(root)	Заглушка начала сканирования: в консольном режиме никаких действий не выполняет

Метод	Назначение
finish_scan(summary)	Заглушка конца сканирования: итоговую сводку печатает функция main
rollback()	Заглушка, вызывается только при ошибке сканирования
close()	Заглушка, необходима для единообразия с DatabaseStorage

Наличие заглушек позволяет функции main не различать типы хранилищ: последовательность вызовов begin_scan → run_scan → finish_scan → close одинакова в обоих режимах.

5.5 Класс DatabaseStorage

Класс DatabaseStorage — хранилище, которое записывает результаты в PostgreSQL. При создании объекта выполняется подключение к базе по строке DB_DSN; все операции выполняются в одной транзакции, которая фиксируется методом close или отменяется методом rollback. Атрибуты класса приведены в таблице 10, методы — в таблице 11.

Таблица 10 – Атрибуты класса DatabaseStorage

Атрибут	Тип	Назначение
conn	psycopg2.connection	Соединение с PostgreSQL
scan_id	Optional[int]	Идентификатор текущей строки в таблице scans
root	Optional[Path]	Абсолютный путь корневой папки скана
path_cache	dict[str, int]	Кэш соответствия «полный путь папки → id в таблице paths», сокращает число запросов

Таблица 11 – Методы класса DatabaseStorage

Метод	Назначение	Возвращаемое значение
__init__()	Открывает соединение psycopg2.connect(DB_DSN), инициализирует scan_id, root и путь_кэш	—
begin_scan(root)	Запоминает абсолютный путь корня и вставляет строку в scans; started_at заполняется сервером	—
get_path_id(directory)	Ищет папку в paths или создаёт её; для некорневых папок рекурсивно создаёт родительскую; использует кэш; вставка выполняется с ON CONFLICT (full_path)	int
save(record)	Сохраняет одну запись: помечает удалённый файл статусом deleted либо вставляет или обновляет строку в files (ON CONFLICT (path_id, name)) и пишет JSON-запись в таблицу json	—
finish_scan(summary)	Записывает в scans время окончания и счётчики по статусам	—

Метод	Назначение	Возвращаемое значение
rollback()	Отменяет незавершённую транзакцию и закрывает соединение	—
close()	Фиксирует изменения (commit) и закрывает соединение	—

Метод get_path_id строит иерархию папок сверху вниз: для папки, не являющейся корнем скана, сначала рекурсивно определяется идентификатор родительской папки, затем вставляется текущая. Для корня диска, у которого имя пустое, в поле name записывается полный путь. Результат кэшируется в словаре path_cache, поэтому повторные обращения к одной папке не приводят к запросам к базе данных.

Метод save для существующего файла обновляет расширение, размер, MD5, даты, содержимое, статус и last_scan_id, а поле first_scan_id не изменяет, сохраняя скан, в котором файл был обнаружен впервые. Счётчик change_count увеличивается на единицу, если значение modified_at в базе отличается от нового значения (условие files.modified_at IS DISTINCT FROM EXCLUDED.modified_at).

5.6 Функции уровня модуля

Помимо классов, в программе объявлены девять функций уровня модуля. Каждая функция выполняет отдельную задачу и может вызываться из других функций. Перечень функций приведён в таблице 12.

Таблица 12 – Функции уровня модуля

Функция	Назначение	Возвращаемое значение
compute_md5(path)	Вычисляет контрольную сумму MD5 файла потоковым чтением блоками по 64 КБ	str из 32 символов
convert_office_file(path)	Конвертирует doc в docx и xls в xlsx средствами Microsoft Office	Optional[Path]
read_text_content(path, size_bytes)	Извлекает текст или метаданные в зависимости от расширения	Optional[str]
build_record(path)	Формирует запись FileRecord по сведениям о файле	Optional[FileRecord]
load_snapshot()	Читает снимок прошлого скана	dict[str, str]

Функция	Назначение	Возвращаемое значение
save_snapshot(records)	Записывает снимок текущего скана в snapshot.json	None
scan_directory(root)	Рекурсивно обходит каталог и отдаёт записи по одной (генератор)	Iterator[FileRecord]
run_scan(root)	Выполняет основной проход: определяет статусы и сохраняет записи	list[FileRecord]
main()	Точка входа: проверяет каталог, управляет транзакцией, печатает сводку	None

5.7 Функция compute_md5

Функция compute_md5(path: Path) -> str вычисляет контрольную сумму MD5 файла. Файл открывается в бинарном режиме и читается блоками по HASH_CHUNK_SIZE (64 КБ); объект hashlib.md5 обновляется на каждом прочитанном блоке, поэтому файл не загружается в оперативную память целиком. Функция возвращает строку из 32 шестнадцатеричных символов в нижнем регистре.

Именно по этому значению файл сравнивается со снимком прошлого скана и определяется его статус. Функция не перехватывает исключения: ошибка чтения (OSError) обрабатывается в функции build_record.

5.8 Функция convert_office_file

Функция convert_office_file(path: Path) -> Optional[Path] конвертирует устаревшие форматы Office в современные: doc в docx и xls в xlsx. Конвертация выполняется через установленный Microsoft Office с помощью библиотеки pywin32 (COM): приложение Word или Excel запускается в фоновом режиме (Visible = False, DisplayAlerts = 0), открывает исходный файл и сохраняет рядом его копию в новом формате (SaveAs2 с кодом 16 для Word, SaveAs с кодом 51 для Excel), после чего приложение закрывается в блоке finally.

Если расширение файла не doc и не xls, функция возвращает None. Если конвертированная копия уже существует рядом с исходным файлом, возвращается путь к ней без повторной конвертации. При возникновении

ошибки выводится сообщение [CONVERT-SKIP], а функция возвращает None, не прерывая работу программы.

5.9 Функция read_text_content

Функция read_text_content(path: Path, size_bytes: int) -> Optional[str] извлекает текст или метаданные из файла и является основной функцией извлечения содержимого. Если размер файла превышает MAX_CONTENT_SIZE (5 МБ), функция сразу возвращает None, не обрабатывая файл. Далее в зависимости от расширения выбирается способ чтения, приведённый в таблице 13.

Таблица 13 – Способы извлечения содержимого по расширениям

Расширение	Способ обработки	Результат
.txt, .py, .json, .md, .log, .xml	Файл читается как набор байтов, кодировка определяется chardet, декодирование выполняется с параметром errors="replace", символ NUL удаляется	текст файла
.docx	Извлекается текст всех абзацев через Document	текст абзацев через перевод строки
.xlsx	Открывается в режимах read_only и data_only, перебираются все листы и строки	значения ячеек через табуляцию
.pptx	Перебираются слайды и элементы с текстовой рамкой	заголовок «--- Слайд N ---» и текст элементов
.pdf	Открывается через fitz, текст извлекается постранично	текст всех страниц
.csv	Определяется кодировка, данные разбираются csv.reader	строки, колонки через табуляцию
.yaml, .yml	Безопасный разбор yaml.safe_load и обратная сериализация safe_dump	нормализованный YAML
.html, .htm	Разбор BeautifulSoup, извлечение только текста	текст без разметки
.odt, .ods, .odp	Загрузка load_odf и извлечение teletype.extractText	текст документа
.jpg, .jpeg, .png, .gif, .bmp, .tiff, .webp	Открытие через PIL Image	строка с форматом, размерами и цветовой моделью
.mp3, .flac, .ogg, .m4a, .wav, .wma	Чтение тегов через MutagenFile с easy=True	пары «ключ: значение»
.zip / .7z / .rar	Открытие архива соответствующей библиотекой	список файлов внутри архива
.sqlite, .db	Подключение sqlite3 и запрос к sqlite_master	список таблиц базы
.doc, .xls	Сначала convert_office_file, затем чтение как docx или xlsx	текст документа или значения ячеек
.ppt	Проверка olefile.isOleFile и перебор потоков	список внутренних потоков OLE-контейнера
прочие	Создание парсера hachoir и извлечение метаданных	словарь метаданных в виде строки

При любой ошибке выводится сообщение [CONTENT-SKIP], а функция возвращает None. Если формат файла неизвестен и метаданные извлечь не удалось, также возвращается None. Значение None соответствует пустому полю content в базе данных.

5.10 Функция build_record

Функция build_record(path: Path) -> Optional[FileRecord] формирует запись о файле. С помощью path.stat() получаются сведения о файле (размер, даты), через compute_md5 вычисляется контрольная сумма, через read_text_content извлекается содержимое, а даты создания и изменения переводятся в UTC функцией datetime.fromtimestamp с параметром tz=timezone.utc. Статус на этом этапе устанавливается равным FileStatus.NEW и уточняется позже в функции run_scan.

Если файл не удаётся прочитать (возникает OSError, в том числе PermissionError), выводится сообщение [SKIP], и функция возвращает None: запись о таком файле не создаётся и в снимок он не попадает.

5.11 Функция load_snapshot

Функция load_snapshot() -> dict[str, str] читает снимок прошлого скана из файла SNAPSHOT_FILE. Если файла нет (первый запуск), возвращается пустой словарь, и все файлы считаются новыми. Если файл существует, из него читается JSON, и формируется словарь, где ключ — полный путь, значение — контрольная сумма MD5. Если JSON повреждён или не читается, выводится предупреждение [WARN], и возвращается пустой словарь: программа считает, что снимка нет.

5.12 Функция save_snapshot

Функция save_snapshot(records: list[FileRecord]) -> None сохраняет снимок текущего скана в файл snapshot.json. Записи со статусом DELETED в снимок не включаются: они отфильтровываются по условию status != FileStatus.DELETED. JSON записывается с параметрами ensure_ascii=False

(кириллица сохраняется в читаемом виде) и indent=2 (с отступами) в кодировке UTF-8; файл перезаписывается целиком.

5.13 Функция scan_directory

Функция scan_directory(root: Path) -> Iterator[FileRecord] является генератором: она рекурсивно обходит каталог с помощью root.rglob("*"), пропускает элементы, не являющиеся файлами, формирует запись через build_record и отдаёт её наружу по одной с помощью yield, не собирая все записи в памяти. Если build_record вернул None, запись не отдаётся.

5.14 Функция run_scan

Функция run_scan(root: Path) -> list[FileRecord] выполняет основной проход сканирования. Сначала загружается прошлый снимок, затем заводятся множество встреченных путей seen_paths и список результатов results. Для каждого файла путь помечается как увиденный, статус определяется по правилам таблицы 14, после чего запись сохраняется через storage.save и добавляется в список.

Таблица 14 – Правила определения статуса файла

Условие	Статус	Пояснение
Путь отсутствует в снимке	new	Файл встречен впервые либо снимок отсутствует
Путь есть в снимке, MD5 совпал	unchanged	Содержимое не изменилось
Путь есть в снимке, MD5 отличается	modified	Содержимое изменено
Путь есть в снимке, файла нет на диске	deleted	Формируется запись-маркер: размер 0, старый MD5 из снимка, текущие даты

После обхода файлов выполняется поиск удалённых: все пути из снимка, которых не было в текущем скане, превращаются в записи-маркеры со статусом DELETED и также сохраняются через хранилище. Функция возвращает список всех записей, включая маркеры удалённых файлов.

5.15 Функция main

Функция main() -> None является точкой входа. Сначала проверяется, что каталог ROOT_PATH существует; при отсутствии каталога выводится сообщение «Путь не существует: <путь>» и работа завершается без создания записи о скане. Далее в блоке try выполняются вызовы storage.begin_scan, run_scan, подсчёт сводки summary по статусам, storage.finish_scan и storage.close. При возникновении любого исключения вызывается storage.rollback, а исключение пробрасывается дальше. После успешного завершения вызывается save_snapshot, и в консоль выводится итоговая сводка.

5.16 Порядок взаимодействия функций и классов

При одном запуске программы вызовы выполняются в следующей последовательности:

1) main() проверяет существование ROOT_PATH;

2) storage.begin_scan(root) создаёт строку в таблице scans (в консольном режиме — заглушка);

3) run_scan(root) вызывает load_snapshot() и получает словарь «путь → MD5»;

4) scan_directory(root) перебирает файлы и для каждого вызывает build_record(path);

5) build_record(path) вызывает compute_md5(path) и read_text_content(path, size); read_text_content при необходимости вызывает convert_office_file(path);

6) run_scan сравнивает полученный MD5 со снимком, устанавливает статус и вызывает storage.save(record);

7) storage.save(record) вызывает get_path_id(каталог файла) и записывает строки в files и json;

8) run_scan формирует записи-маркеры для удалённых файлов и сохраняет их;

9) main() подсчитывает сводку и вызывает storage.finish_scan(summary);

10) storage.close() фиксирует транзакцию; save_snapshot(results) перезаписывает snapshot.json; в консоль выводится сводка.

6 ТЕСТОВЫЕ ПРИМЕРЫ

6.1 Назначение и методика тестирования

Тестовые примеры предназначены для проверки соответствия фактического поведения программы требованиям, заложенным в её исходном тексте. Примеры построены по принципу «чёрного ящика»: проверяется реакция программы на состояние файловой системы и содержимого файлов, а не внутренние структуры данных. Каждый пример соотнесён с конкретными функциями и методами, реализующими проверяемое поведение, что позволяет локализовать причину расхождения.

Тестовые примеры составлены для программы в поставляемой конфигурации. При выполнении на другом рабочем месте допускается изменять только значения констант конфигурации (ROOT_PATH, DB_DSN, ENABLE_DB_SYNC), как это предусмотрено инструкцией по установке; логика программы и состав функций при этом не изменяются.

Порядок проведения испытаний:

1) подготовить тестовый набор файлов в каталоге, указанном в ROOT_PATH;

2) убедиться, что файл snapshot.json отсутствует, а база данных содержит только что созданные пустые таблицы;

3) выполнять примеры в порядке их следования: примеры образуют цепочку, в которой каждый следующий использует состояние, сформированное предыдущим;

4) после каждого запуска фиксировать вывод в консоли и состояние таблиц scans, paths, files, json и файла snapshot.json;

5) сравнить фактический результат с ожидаемым и проставить отметку о соответствии.

Если пример требует изменения константы ROOT_PATH или ENABLE_DB_SYNC, это указано в графе «Исходные данные»; после выполнения такого примера значение константы восстанавливается.

6.2 Тестовый набор

Тестовый набор размещается в каталоге, заданном параметром ROOT_PATH (в примерах принято значение E:\MAX_files). Состав набора приведён в таблице 15. Набор подобран так, чтобы задействовать основные ветки функции read_text_content, проверку предельного размера, обработку ошибок доступа и все четыре статуса файлов.

Таблица 15 – Состав тестового набора

№	Файл	Размер	Формат	Что проверяется
1	docs\report.txt	115 Б	текст UTF-8, кириллица	Ветка TEXT_EXTENSIONS, извлечение текста, статусы new и modified
2	docs\notes.md	41 Б	Markdown	Ветка TEXT_EXTENSIONS для расширения .md
3	docs\legacy_1251.txt	36 Б	текст Windows-1251	Автоматическое определение кодировки chardet
4	docs\act.docx	около 36 КБ	Word	Ветка .docx, извлечение текста абзацев
5	tables\data.csv	30 Б	CSV с разделителем «;»	Ветка .csv, разбор csv.reader
6	media\logo.png	260 Б	PNG 120 × 80	Ветка изображений, определение формата и размеров
7	big\huge.bin	6 291 456 Б	двоичный файл 6 МБ	Ограничение MAX_CONTENT_SIZE: содержимое не извлекается
8	arch\sample.zip	около 300 Б	ZIP с двумя файлами	Ветка .zip, получение списка файлов архива
9	db\sample.sqlite	около 12 КБ	SQLite, две таблицы	Ветка .sqlite, чтение списка таблиц
10	locked\no_access.txt	20 Б	текст без прав чтения	Обработка OSError: сообщение [SKIP], запись не создаётся

Файл locked\no_access.txt готовится путём назначения прав доступа, запрещающих чтение текущему пользователю (в свойствах файла на вкладке «Безопасность» снимаются все разрешения). Для файла docs\act.docx достаточно создать документ Word с одной строкой текста; для архива sample.zip — поместить в архив два текстовых файла; для базы sample.sqlite — создать две таблицы, например users и orders.

6.3 Тестовые примеры

ТП-1. Первый запуск при отсутствии файла снимка

Параметр	Содержание
Проверяемые функции	main, load_snapshot, run_scan, scan_directory, build_record, compute_md5, read_text_content, DatabaseStorage.begin_scan, save, finish_scan, close, save_snapshot
Исходные данные	Тестовый набор из таблицы 15; файл snapshot.json удалён; таблицы базы данных пусты; ENABLE_DB_SYNC = True; ROOT_PATH = E:\MAX_files
Действия	1) удалить файл snapshot.json, если он существует; 2) выполнить в командной строке python main.py; 3) дождаться вывода итоговой сводки
Ожидаемый результат	Для девяти файлов выведены строки [НОВЫЙ] с полными путями; для файла locked\no_access.txt выведено сообщение [SKIP] с указанием PermissionError; в сводке: «Всего обработано: 9», «новых: 9», остальные счётчики равны нулю. В таблице scans одна строка с заполненным finished_at и files_total = 9; в таблице files девять строк со status = 'new', first_scan_id = last_scan_id = id скана, change_count = 0; в таблице json девять строк; в таблице paths семь строк; файл snapshot.json содержит девять записей
Критерий приёмки	Количество записей в files и snapshot.json равно числу читаемых файлов набора; статусы всех записей равны new; счётчики сводки совпадают с ожидаемыми

Пояснение: файл locked\no_access.txt не попадает ни в files, ни в snapshot.json, так как при получении сведений возникает исключение PermissionError, которое перехватывается в build_record. Число папок в paths меньше числа каталогов на диске: строки создаются только для каталогов, содержащих файлы, и для их предков.

ТП-2. Повторный запуск без изменений

Параметр	Содержание
Проверяемые функции	load_snapshot, run_scan (сравнение MD5), DatabaseStorage.save (ветка ON CONFLICT), finish_scan
Исходные данные	Состояние после ТП-1: файл snapshot.json существует и содержит девять записей; содержимое файлов не изменялось
Действия	1) не изменяя файлов, выполнить python main.py; 2) проверить сводку и таблицу files
Ожидаемый результат	Для девяти файлов выведены строки [БЕЗ ИЗМЕНЕНИЙ]; в сводке: «Всего обработано: 9», «без изменений: 9», остальные счётчики равны нулю. В таблице scans появилась вторая строка с files_unchanged = 9; в таблице files по-прежнему девять строк, значения first_scan_id не изменились, change_count остался равным нулю; файл snapshot.json перезаписан
Критерий приёмки	Ни одна запись не получила статус new или modified; change_count не увеличился, так как значение modified_at не изменилось

ТП-3. Изменение содержимого файла

Параметр	Содержание
Проверяемые функции	compute_md5, run_scan (ветка MODIFIED), DatabaseStorage.save (увеличение change_count)
Исходные данные	Состояние после ТП-2; в файл docs\report.txt дописана одна строка
Действия	1) дописать строку в docs\report.txt и сохранить файл; 2) выполнить python main.py
Ожидаемый результат	Для report.txt выведена строка [ИЗМЕНЁН], для остальных восьми файлов — [БЕЗ ИЗМЕНЕНИЙ]; в сводке «изменено: 1», «без изменений: 8». В таблице files у записи report.txt status = 'modified', change_count = 1, изменились size_bytes, md5_hash и modified_at; в таблице json обновились поле data и updated_at
Критерий приёмки	Статус modified присвоен только изменённому файлу; change_count равен единице; новое значение md5_hash отличается от значения в предыдущем снимке

ТП-4. Изменение только даты изменения файла

Параметр	Содержание
Проверяемые функции	run_scan (сравнение по MD5), DatabaseStorage.save (условие проверки modified_at)
Исходные данные	Состояние после ТП-3; содержимое docs\notes.md не меняется, обновляется только дата последнего изменения
Действия	1) выполнить в PowerShell команду (Get-Item 'E:\MAX_files\docs\notes.md').LastWriteTime = Get-Date; 2) выполнить python main.py
Ожидаемый результат	Для notes.md выведена строка [БЕЗ ИЗМЕНЕНИЙ], так как контрольная сумма MD5 не изменилась. В таблице files у записи notes.md статус остался 'unchanged', но change_count увеличился до единицы, а поле modified_at содержит новое значение
Критерий приёмки	Статус определяется по MD5, а не по дате; при этом счётчик изменений реагирует на изменение modified_at независимо от статуса

ТП-5. Извлечение текста в кодировке Windows-1251

Параметр	Содержание
Проверяемые функции	read_text_content (ветка текстовых файлов), chardet.detect
Исходные данные	Файл docs\legacy_1251.txt сохранён в кодировке Windows-1251 и содержит строку «Старый файл в кодировке Windows-1251»
Действия	1) выполнить запрос SELECT name, content FROM files WHERE name = 'legacy_1251.txt'; 2) сравнить содержимое с исходным текстом
Ожидаемый результат	Поле content содержит читаемый текст «Старый файл в кодировке Windows-1251» без символов замещения; размер файла равен 36 байтам, что соответствует однобайтовой кодировке

Параметр	Содержание
Критерий приёмки	Текст извлечён без искажений; в поле content отсутствуют символы замещения и знаки вопроса, характерные для неверно определённой кодировки

ТП-6. Извлечение содержимого для различных форматов

Параметр	Содержание
Проверяемые функции	read_text_content (ветки docx, csv, изображений, zip, sqlite)
Исходные данные	Файлы тестового набора, перечисленные в таблице 15; состояние после ТП-2 (все файлы имеют статус unchanged)
Действия	1) выполнить запрос SELECT f.name, left(f.content, 80) AS content_head FROM files f ORDER BY f.name; 2) сверить значения с таблицей ожидаемых результатов
Ожидаемый результат	Для act.docx — текст абзаца «Тестовый документ Word»; для data.csv — строки «id;name;price» и «1;Laptop;1000»; для logo.png — строка «Изображение PNG 120x80 mode=RGB»; для sample.zip — имена двух файлов архива; для sample.sqlite — имена двух таблиц; для huge.bin — значение NULL
Критерий приёмки	Для каждого файла получено непустое содержимое соответствующего вида, кроме файла, превышающего предельный размер; сообщения [CONTENT-SKIP] в консоли отсутствуют

ТП-7. Файл, превышающий предельный размер

Параметр	Содержание
Проверяемые функции	read_text_content (проверка MAX_CONTENT_SIZE), compute_md5, build_record
Исходные данные	Файл big\huge.bin размером 6 291 456 байт; дополнительно подготовлены файлы размером ровно 5 242 880 байт и 5 242 881 байт
Действия	1) выполнить запрос SELECT name, size_bytes, content IS NULL AS no_content, length(md5_hash) AS md5_len FROM files WHERE name = 'huge.bin'; 2) сравнить поведение на граничных размерах
Ожидаемый результат	Для huge.bin: no_content = true, md5_len = 32, size_bytes = 6291456; статус файла определяется обычным образом. Файл размером 5 242 880 байт обрабатывается на содержимое, файл размером 5 242 881 байт — нет, так как в коде используется строгое сравнение size_bytes > MAX_CONTENT_SIZE
Критерий приёмки	Содержимое не извлекается только для файлов строго больше 5 МБ; контрольная сумма вычисляется для всех файлов независимо от размера

ТП-8. Появление нового файла во вложенном каталоге

Параметр	Содержание
Проверяемые функции	scan_directory (рекурсивный обход), run_scan (статус new), DatabaseStorage.get_path_id (рекурсивное создание папок)

Параметр	Содержание
Исходные данные	Состояние после ТП-4; создан каталог docs\sub и в нём файл inner.txt
Действия	1) создать каталог docs\sub и поместить в него файл inner.txt с произвольным текстом; 2) выполнить python main.py
Ожидаемый результат	Выведена строка [НОВЫЙ] с путём E:\MAX_files\docs\sub\inner.txt, для остальных файлов — [БЕЗ ИЗМЕНЕНИЙ]; в сводке «Всего обработано: 10», «новых: 1», «без изменений: 9». В таблице paths появилась строка с full_path = E:\MAX_files\docs\sub и parent_id, равным идентификатору каталога docs; в таблице files десять строк
Критерий приёмки	Иерархия каталогов в paths восстановлена корректно; новый файл получил статус new и идентификатор скана в полях first_scan_id и last_scan_id

ТП-9. Переименование файла

Параметр	Содержание
Проверяемые функции	run_scan (поиск удалённых по снимку), DatabaseStorage.save (ветка DELETED)
Исходные данные	Состояние после ТП-8; файл tables\data.csv переименован в tables\prices.csv без изменения содержимого
Действия	1) переименовать файл; 2) выполнить python main.py
Ожидаемый результат	Выведены строка [НОВЫЙ] для prices.csv и строка [УДАЛЁН] для data.csv с прежним путём; в сводке «Всего обработано: 11», «новых: 1», «без изменений: 9», «удалено: 1». В таблице files появилась новая строка для prices.csv, а строка data.csv получила status = 'deleted', новый last_scan_id и обновлённое scanned_at, при этом размер и содержимое строки сохранились
Критерий приёмки	Программа трактует переименование как удаление и появление файла, поскольку ключом снимка является полный путь

ТП-10. Удаление файла с диска

Параметр	Содержание
Проверяемые функции	run_scan (формирование записи-маркера), save_snapshot (фильтрация DELETED), DatabaseStorage.save
Исходные данные	Состояние после ТП-9; файл docs\legacy_1251.txt удалён
Действия	1) удалить файл; 2) выполнить python main.py; 3) проверить файл snapshot.json
Ожидаемый результат	Выведена строка [УДАЛЁН] с путём удалённого файла; в сводке «Всего обработано: 10», «без изменений: 9», «удалено: 1». В таблице files запись имеет status = 'deleted'; в файле snapshot.json девять записей, записи об удалённом файле нет. При следующем запуске сообщение об этом файле не выводится
Критерий приёмки	Строка не удалена из таблицы files, а помечена статусом deleted; в снимок удалённые файлы не попадают

ТП-11. Повреждённый файл снимка

Параметр	Содержание
Проверяемые функции	load_snapshot (обработка JSONDecodeError), run_scan
Исходные данные	Состояние после ТП-10; в файл snapshot.json внесена строка «{ это не json», нарушающая синтаксис
Действия	1) испортить файл snapshot.json; 2) выполнить python main.py
Ожидаемый результат	Первой строкой вывода является сообщение [WARN] «Не удалось прочитать снимок: Expecting property name enclosed in double quotes...»; все девять файлов получают статус new; в сводке «новых: 9»; файл snapshot.json перезаписан корректным содержимым
Критерий приёмки	Программа не завершается аварийно, а работает так же, как при отсутствии снимка

ТП-12. Несуществующий каталог сканирования

Параметр	Содержание
Проверяемые функции	main (проверка root.exists())
Исходные данные	В константе ROOT_PATH указан несуществующий каталог, например D:\no_such_folder; ENABLE_DB_SYNC = True
Действия	1) изменить значение ROOT_PATH; 2) выполнить python main.py; 3) проверить код возврата командой echo %ERRORLEVEL%
Ожидаемый результат	Выведено единственное сообщение «Путь не существует: D:\no_such_folder»; код возврата равен 0; новая строка в таблице scans не появилась, так как проверка выполняется до вызова begin_scan; файл snapshot.json не изменён
Критерий приёмки	Отсутствие каталога не приводит к исключению и не создаёт запись о скане

ТП-13. Работа в консольном режиме

Параметр	Содержание
Проверяемые функции	ConsoleStorage.save, begin_scan, finish_scan, rollback, close; main
Исходные данные	ENABLE_DB_SYNC = False; состояние после ТП-11 (файл снимка корректен)
Действия	1) установить ENABLE_DB_SYNC = False; 2) выполнить python main.py; 3) проверить содержимое базы данных
Ожидаемый результат	Для каждого файла выведена карточка из семи строк: русская подпись статуса и имя файла, технический статус, путь, размер в байтах, MD5, дата создания и дата изменения в местном времени; итоговая сводка выведена; число строк в таблице scans не изменилось; файл snapshot.json обновлён
Критерий приёмки	Подключение к базе данных не выполняется, формат карточки соответствует методу save класса ConsoleStorage

ТП-14. Сканирование пустого каталога

Параметр	Содержание
Проверяемые функции	main, run_scan, finish_scan, save_snapshot
Исходные данные	В ROOT_PATH указан существующий пустой каталог; файл snapshot.json отсутствует; ENABLE_DB_SYNC = True
Действия	1) создать пустой каталог и указать его в ROOT_PATH; 2) выполнить python main.py
Ожидаемый результат	Строки статусов не выводятся; в сводке все счётчики равны нулю; в таблице scans появилась строка с files_total = 0 и заполненным finished_at; таблица paths осталась без изменений; файл snapshot.json содержит пустой массив []
Критерий приёмки	Программа корректно обрабатывает отсутствие файлов: запись о скане создаётся, записи о папках — нет, так как метод get_path_id не вызывается

ТП-15. Файл без прав доступа на чтение

Параметр	Содержание
Проверяемые функции	build_record (перехват OSError), scan_directory
Исходные данные	В ROOT_PATH размещён только файл locked\no_access.txt с запретом чтения; файл snapshot.json отсутствует
Действия	1) снять все разрешения на чтение файла для текущего пользователя; 2) выполнить python main.py
Ожидаемый результат	Выведено сообщение [SKIP] с путём файла и текстом PermissionError; в сводке «Всего обработано: 0»; в таблице files записей нет; файл snapshot.json содержит пустой массив
Критерий приёмки	Ошибка доступа не прерывает сканирование и не создаёт частичную запись о файле

ТП-16. Заполнение таблицы scans и согласованность счётчиков

Параметр	Содержание
Проверяемые функции	DatabaseStorage.begin_scan, finish_scan
Исходные данные	Результаты выполнения ТП-1 — ТП-11: в таблице scans накопилось несколько строк
Действия	1) выполнить запрос SELECT id, files_total, files_new, files_unchanged, files_modified, files_deleted, finished_at IS NOT NULL AS closed FROM scans ORDER BY id; 2) проверить выполнение условия files_total = files_new + files_unchanged + files_modified + files_deleted
Ожидаемый результат	Число строк равно числу выполненных запусков; для всех строк closed = true; для каждой строки сумма четырёх счётчиков равна files_total; значения идентификаторов возрастают

Параметр	Содержание
Критерий приёмки	Счётчики скана согласованы с фактическим числом обработанных записей; незакрытых сканов нет

6.4 Сводные результаты тестовых примеров

Соответствие тестовых примеров проверяемым функциям и ожидаемым итогам приведено в таблице 16. Отметка о соответствии фактических результатов ожидаемым проставляется при проведении испытаний.

Таблица 16 – Сводные результаты тестовых примеров

№	Проверяемое поведение	Проверяемые функции и методы	Ожидаемый итог
1	Первый запуск без снимка	main, load_snapshot, run_scan, begin_scan, save, finish_scan	9 файлов со статусом new; 1 сообщение [SKIP]
2	Повторный запуск	run_scan, save, finish_scan	9 файлов со статусом unchanged
3	Изменение содержимого	compute_md5, run_scan, save	1 modified, change_count = 1
4	Изменение только даты	run_scan, save	статус unchanged, change_count увеличился
5	Кодировка Windows-1251	read_text_content, chardet	текст извлечён без искажений
6	Различные форматы	read_text_content	содержимое соответствует формату файла
7	Файл более 5 МБ	read_text_content	content = NULL, MD5 вычислен
8	Новый файл во вложенном каталоге	scan_directory, run_scan, get_path_id	1 new, иерархия paths дополнена
9	Переименование файла	run_scan, save	1 new и 1 deleted
10	Удаление файла	run_scan, save_snapshot	1 deleted, запись исключена из снимка
11	Повреждённый снимок	load_snapshot	сообщение [WARN], все файлы new
12	Несуществующий каталог	main	сообщение, строка scans не создана
13	Консольный режим	ConsoleStorage	карточки файлов, база не изменяется
14	Пустой каталог	main, run_scan, save_snapshot	нулевые счётчики, снимок []
15	Файл без прав чтения	build_record	сообщение [SKIP], записей нет
16	Счётчики таблицы scans	begin_scan, finish_scan	files_total равен сумме четырёх счётчиков

Тестовые примеры охватывают все четыре статуса файла, обе реализации хранилища, основные ветки извлечения содержимого, граничное условие предельного размера файла и обработку трёх видов ошибок: отсутствия каталога, повреждения снимка и отказа доступа к файлу.